

Decisiones de diseño BEM: Hotel Vista Verde

Isaac · 5 de octubre de 2026

1. Qué es BEM y por qué se eligió

BEM (Bloque, Elemento, Modificador) es una convención para nombrar las clases de CSS. Se eligió para la página del hotel porque hace que el nombre de cada clase diga qué es y a qué componente pertenece, sin necesidad de abrir el HTML para entenderlo.

La convención usa tres piezas:

- **Bloque:** un componente independiente que se puede reutilizar, como `tarjeta` o `menu`.
- **Elemento:** una parte que solo tiene sentido dentro de su bloque, y se escribe con dos guiones bajos: `tarjeta__titulo`.
- **Modificador:** una variación de un bloque o de un elemento, y se escribe con dos guiones medios: `tarjeta--destacada`.

La decisión de fondo fue que cada pieza de la página tuviera su propia clase y que el CSS dependiera de esa clase, no de la posición de la etiqueta en el documento.

2. Bloques, elementos y modificadores de la página

La página se dividió en los siguientes bloques:

Bloque	Elementos	Modificadores
encabezado	<code>__titulo</code> , <code>__frase</code> , <code>__accion</code>	—
menu	<code>__item</code> , <code>__enlace</code>	—
seccion	<code>__titulo</code>	<code>--habitaciones</code>
medios	<code>__foto</code> , <code>__video</code>	—
fila	—	—
tarjeta	<code>__icono</code> , <code>__titulo</code> , <code>__texto</code> , <code>__foto</code> , <code>__precio</code> , <code>__lista</code> , <code>__item</code> , <code>__avatar</code> , <code>__cargo</code>	<code>--destacada</code> , <code>--testimonio</code>
formulario	<code>__etiqueta</code> , <code>__campo</code> , <code>__boton</code>	—
boton	—	<code>--principal</code>

Bloque	Elementos	Modificadores
pie	__contenedor, __columna, __titulo, __lista, __item, __enlace, __copyright	—

3. Decisiones de diseño y su justificación

3.1 Una sola clase por regla, sin selectores largos

En la primera versión del CSS se usaban selectores como `.menu li a` o reglas directas sobre `div`, `h2` e `img`. Con BEM esas reglas se reemplazaron por clases únicas:

```
/* antes: depende de que el enlace esté dentro de un li dentro de .menu */
.menu li a { color: white; }

/* ahora: la clase basta, sin importar dónde esté el enlace */
.menu__enlace { color: white; }
```

La justificación es la especificidad. Una regla de una sola clase tiene la misma fuerza que las demás, así que no hay conflictos entre reglas y el orden de escritura es predecible. Además, si mañana el enlace se mueve a otro lugar del HTML, el estilo sigue funcionando.

3.2 Evitar reglas globales sobre etiquetas

La regla `div { text-align: center; }` centraba el contenido de todos los `div` de la página, incluso los que no debían centrarse. Se eliminó y el centrado se aplicó donde corresponde, por ejemplo en `.tarjeta` y `.formulario`. De esta forma un cambio en un componente no afecta por accidente a otro.

Solo se dejó una regla sobre etiqueta, la de `body`, porque define el fondo y la tipografía de toda la página y es una configuración base, no un componente.

3.3 Reutilizar el bloque `tarjeta` con modificadores

Las características, las habitaciones y los testimonios comparten la misma estructura: una caja con fondo, borde, sombra y contenido. En lugar de crear tres bloques distintos, se creó un solo bloque `tarjeta` y se usaron modificadores para las variaciones:

```
<div class="tarjeta">...</div>                                <!-- tarjeta normal ->
->
<div class="tarjeta tarjeta--destacada">...</div>            <!-- plan recomendado
-->
<div class="tarjeta tarjeta--testimonio">...</div>            <!-- testimonio -->
```

La clase base aporta el diseño común y el modificador solo agrega o cambia lo que es distinto. Así el diseño base se escribe una vez y las variaciones piden pocas líneas, que era además lo que pedía la tarea al hablar de una variación destacada y de diferentes variaciones de diseño.

3.4 Separar la estructura de la apariencia: el bloque `fila`

El bloque `fila` solo se encarga de poner elementos en horizontal con `display: flex`. No tiene colores ni bordes. Se mantuvo separado de `tarjeta` porque la disposición y la apariencia son responsabilidades distintas, y así la misma `fila` sirve para características, habitaciones y testimonios.

3.5 El botón como bloque propio

El call-to-action no se nombró como parte del encabezado sino como un bloque independiente, `boton`, con el modificador `boton--principal`. Un botón puede aparecer en cualquier parte de la página, por lo que no debe depender de un contenedor concreto. El `div` `encabezado__accion` solo se encarga de centrarlo y de dar el espacio alrededor.

3.6 El `id` solo para los enlaces internos

Los atributos `id` (`#reserva`, `#habitaciones`, `#contacto`) se conservaron únicamente para que el menú pueda saltar a cada sección. No se usan para dar estilos. De esa forma los estilos dependen siempre de clases y el `id` cumple una sola función.

3.7 Convivencia con la estructura semántica

BEM no reemplaza a las etiquetas semánticas (`header`, `nav`, `main`, `section`, `footer`): las dos cosas se complementan. La etiqueta indica el significado de la parte para el navegador y los lectores de pantalla, y la clase BEM indica a qué componente de diseño pertenece, como en `<section class="seccion">`.

3.8 CSS separado del HTML

Ningún estilo se escribió dentro del HTML con el atributo `style`. Todo el diseño vive en `style.css`, de modo que cambiar un color o un tamaño se hace en un solo lugar.

4. Ventajas observadas

- **Legibilidad:** al leer `tarjeta__precio` se entiende qué es y dónde va sin buscar.
- **Mantenimiento:** cada componente se puede cambiar sin riesgo de afectar a otros.
- **Reutilización:** una tarjeta, un botón o una fila se usan en varias secciones.

- **Trabajo en equipo:** otra persona puede entender la convención rápido y agregar componentes con la misma lógica.

5. Desventajas y cómo se manejaron

BEM también tiene costos:

- Los nombres de clase son más largos, y el HTML queda más cargado que con selectores de etiqueta. Se aceptó porque la claridad compensa la longitud.
- Hay que mantener la disciplina de nombrar bien cada pieza; un nombre inconsistente rompe la ventaja de la convención.
- No conviene encadenar elementos como `tarjeta__lista__item`. Cuando una parte pertenece al mismo bloque, se nombra directamente como `tarjeta__item`, sin anidar niveles.

6. Conclusión

Aplicar BEM a la página del hotel dejó un CSS más predecible, más fácil de leer y con menos repetición. Las decisiones más importantes fueron usar una sola clase por regla, eliminar los estilos globales sobre etiquetas, reutilizar el bloque `tarjeta` con modificadores y separar por completo el HTML del CSS. Con esto la página puede crecer, por ejemplo con la sección de precios, agregando componentes nuevos o variaciones sin reescribir lo que ya funciona.